

Object Oriented Programming

Lab Assignment # 1

Submitted To: Muhammad Shahid Bhatti

Submitted By: Qirat Hafsa

Registration # SP26-BAI-041

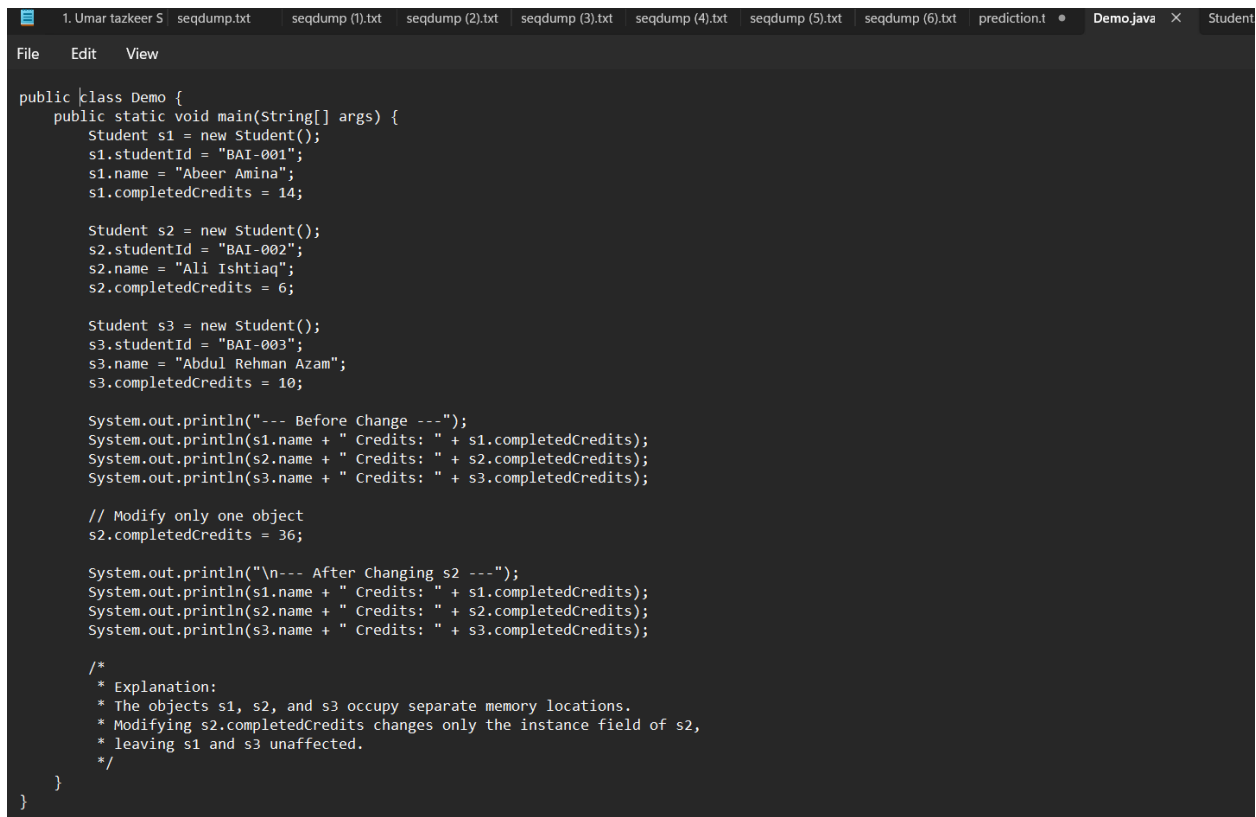
Task 1

Code Picture:



A screenshot of an IDE window titled 'Demo.java'. The window has tabs for 'seqdump (1)' through 'seqdump (6)', 'predic', and 'Demo.java'. The menu bar shows 'File', 'Edit', and 'View'. The code defines a 'Student' class with three attributes: 'studentId' (String), 'name' (String), and 'completedCredits' (int).

```
public class Student{
String studentId;
String name;
int completedCredits;
}
```



A screenshot of an IDE window titled 'Demo.java'. The window has tabs for '1. Umar tazkeer S', 'seqdump.txt', 'seqdump (1).txt', 'seqdump (2).txt', 'seqdump (3).txt', 'seqdump (4).txt', 'seqdump (5).txt', 'seqdump (6).txt', 'prediction.t', 'Demo.java', and 'Student'. The menu bar shows 'File', 'Edit', and 'View'. The code defines a 'Demo' class with a 'main' method. It creates three 'Student' objects (s1, s2, s3) and prints their details. It then modifies the 'completedCredits' of s2 and prints the details again. A comment explains that modifying s2's field only affects s2.

```
public class Demo {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.studentId = "BAI-001";
        s1.name = "Abeer Amina";
        s1.completedCredits = 14;

        Student s2 = new Student();
        s2.studentId = "BAI-002";
        s2.name = "Ali Ishtiaq";
        s2.completedCredits = 6;

        Student s3 = new Student();
        s3.studentId = "BAI-003";
        s3.name = "Abdul Rehman Azam";
        s3.completedCredits = 10;

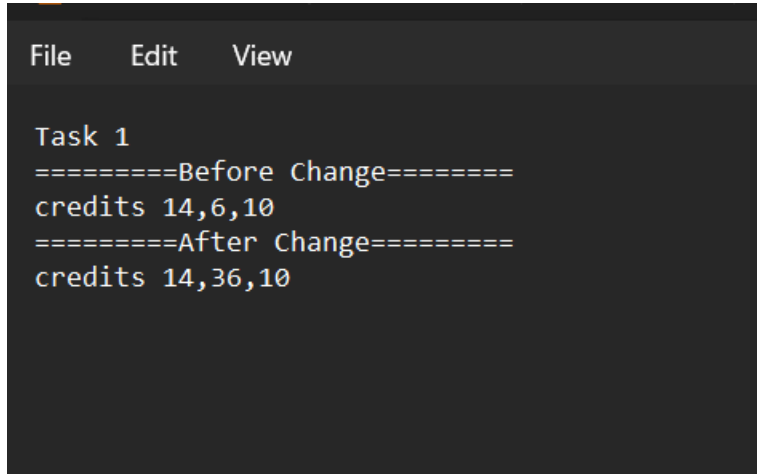
        System.out.println("--- Before Change ---");
        System.out.println(s1.name + " Credits: " + s1.completedCredits);
        System.out.println(s2.name + " Credits: " + s2.completedCredits);
        System.out.println(s3.name + " Credits: " + s3.completedCredits);

        // Modify only one object
        s2.completedCredits = 36;

        System.out.println("\n--- After Changing s2 ---");
        System.out.println(s1.name + " Credits: " + s1.completedCredits);
        System.out.println(s2.name + " Credits: " + s2.completedCredits);
        System.out.println(s3.name + " Credits: " + s3.completedCredits);

        /*
         * Explanation:
         * The objects s1, s2, and s3 occupy separate memory locations.
         * Modifying s2.completedCredits changes only the instance field of s2,
         * leaving s1 and s3 unaffected.
         */
    }
}
```

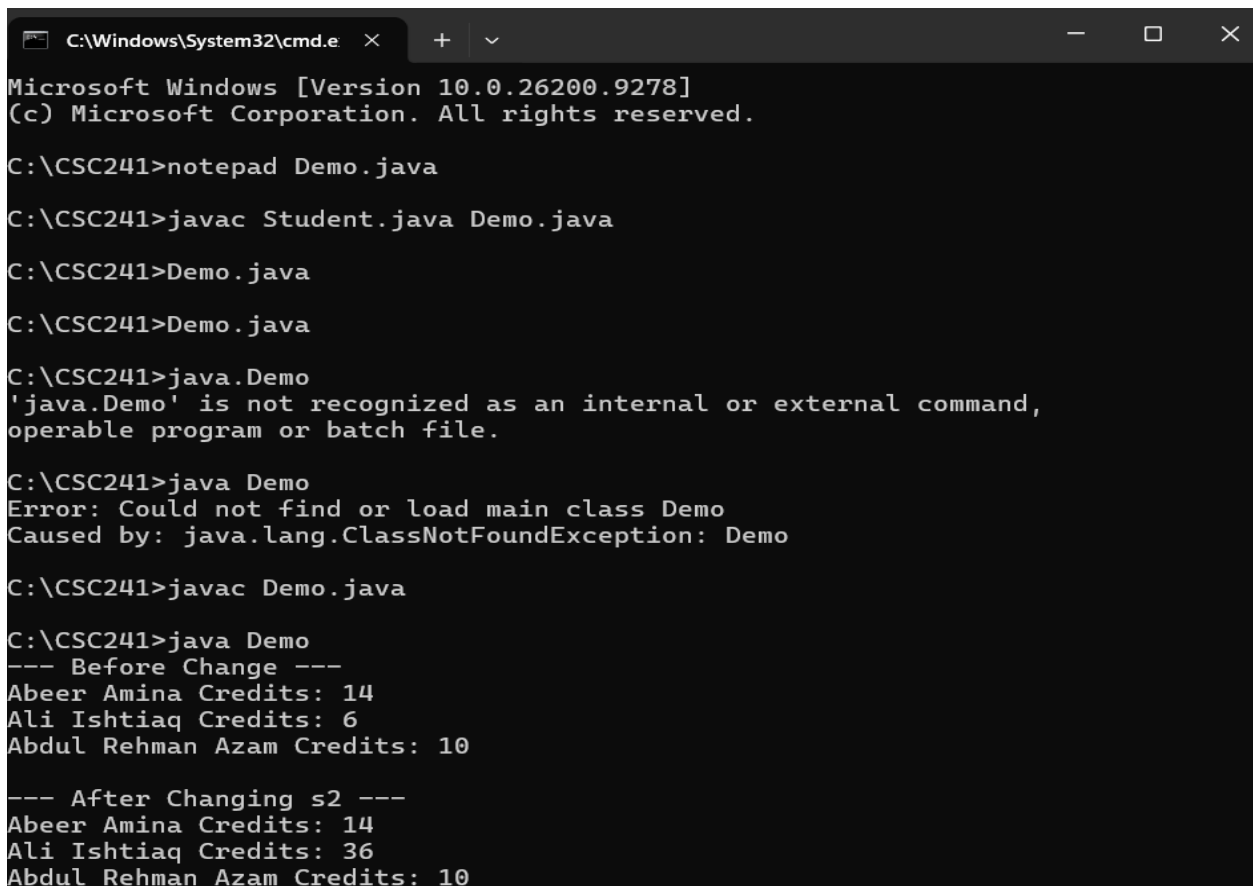
Prediction:



```
File Edit View

Task 1
=====Before Change=====
credits 14,6,10
=====After Change=====
credits 14,36,10
```

Output:



```
C:\Windows\System32\cmd.e  x + v
Microsoft Windows [Version 10.0.26200.9278]
(c) Microsoft Corporation. All rights reserved.

C:\CSC241>notepad Demo.java

C:\CSC241>javac Student.java Demo.java

C:\CSC241>Demo.java

C:\CSC241>Demo.java

C:\CSC241>java.Demo
'java.Demo' is not recognized as an internal or external command,
operable program or batch file.

C:\CSC241>java Demo
Error: Could not find or load main class Demo
Caused by: java.lang.ClassNotFoundException: Demo

C:\CSC241>javac Demo.java

C:\CSC241>java Demo
--- Before Change ---
Abeer Amina Credits: 14
Ali Ishtiaq Credits: 6
Abdul Rehman Azam Credits: 10

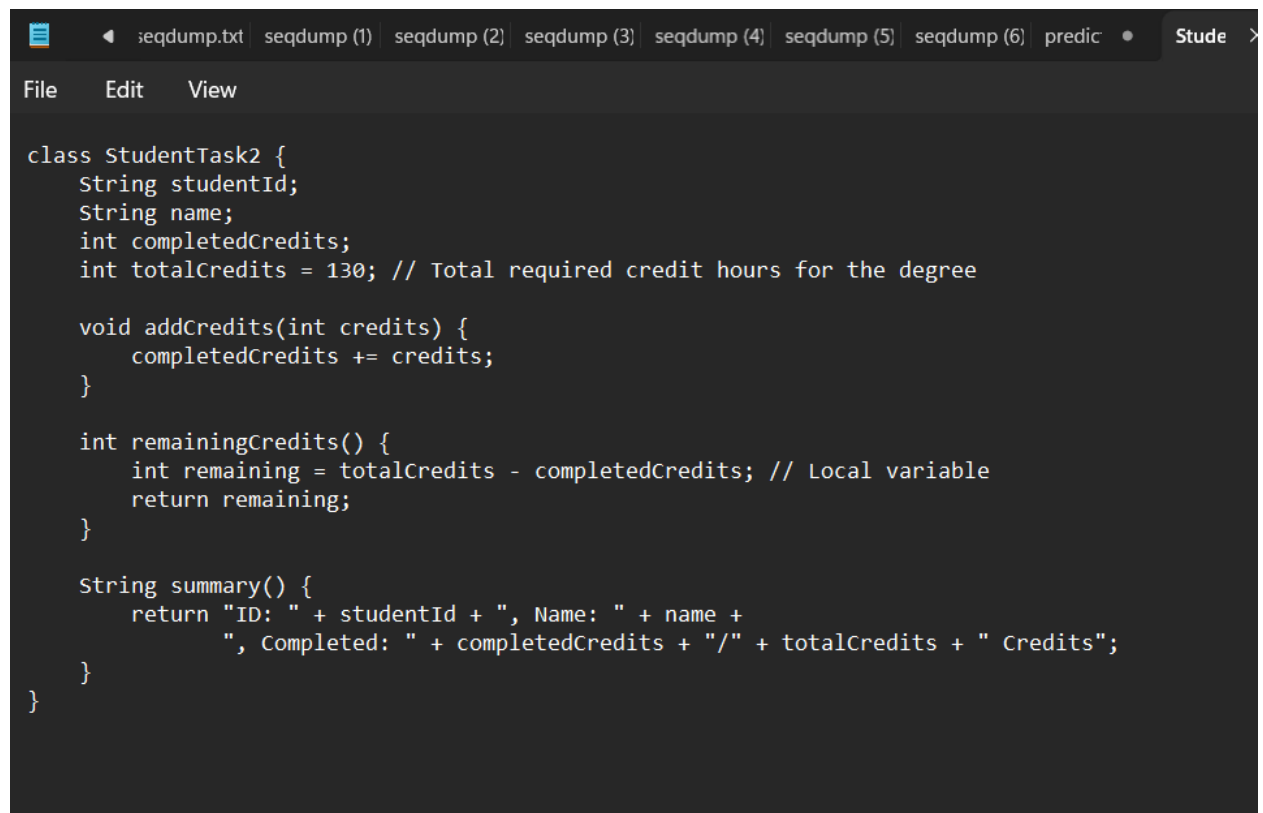
--- After Changing s2 ---
Abeer Amina Credits: 14
Ali Ishtiaq Credits: 36
Abdul Rehman Azam Credits: 10
```

Concepts Answer

Effect of Updating Object Fields: s1 and s3 remain completely unchanged because each variable points to a distinct object instance in memory with its own independent set of instance variables.

Task 2

Code Picture:



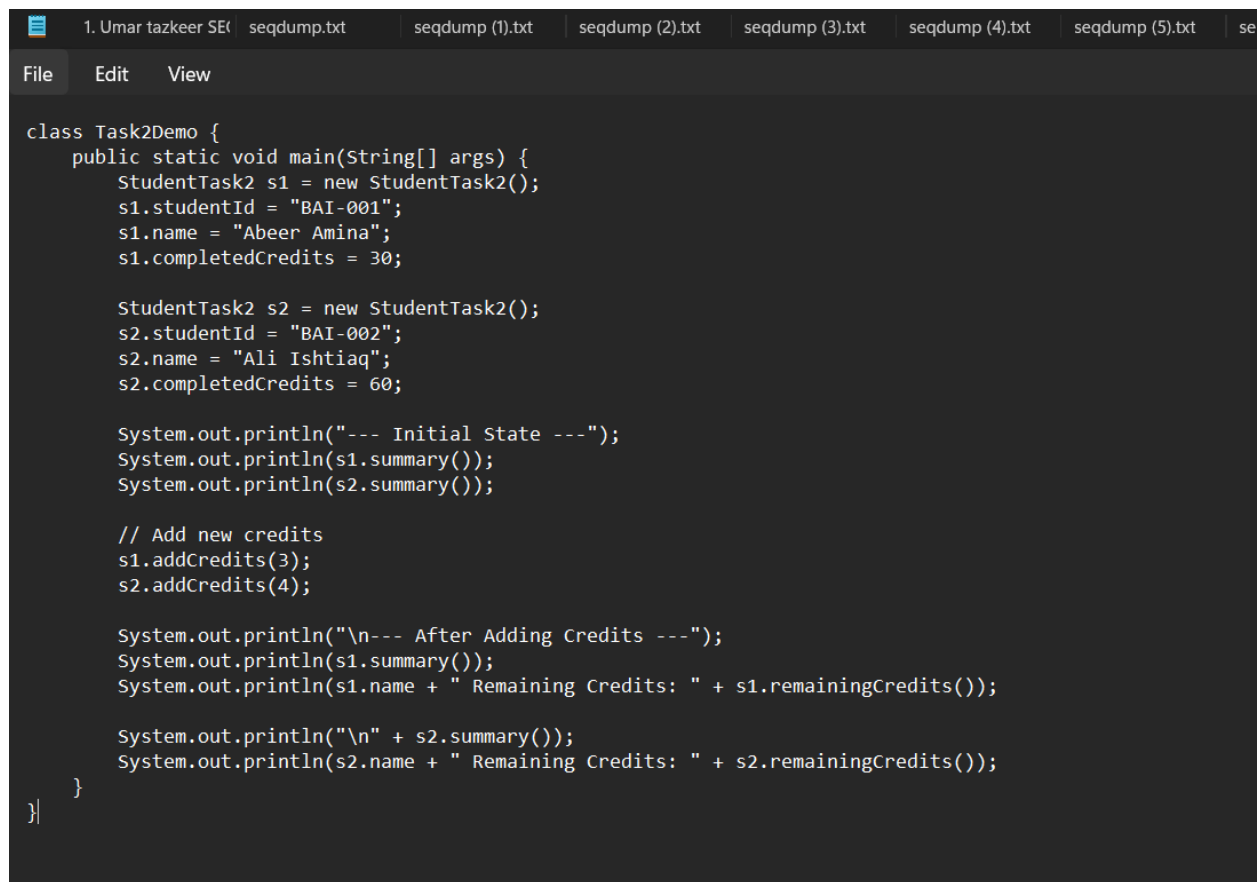
```
seqdump.txt seqdump (1) seqdump (2) seqdump (3) seqdump (4) seqdump (5) seqdump (6) predic • Stude >
File Edit View

class StudentTask2 {
    String studentId;
    String name;
    int completedCredits;
    int totalCredits = 130; // Total required credit hours for the degree

    void addCredits(int credits) {
        completedCredits += credits;
    }

    int remainingCredits() {
        int remaining = totalCredits - completedCredits; // Local variable
        return remaining;
    }

    String summary() {
        return "ID: " + studentId + ", Name: " + name +
            ", Completed: " + completedCredits + "/" + totalCredits + " Credits";
    }
}
```



```
class Task2Demo {
    public static void main(String[] args) {
        StudentTask2 s1 = new StudentTask2();
        s1.studentId = "BAI-001";
        s1.name = "Abeer Amina";
        s1.completedCredits = 30;

        StudentTask2 s2 = new StudentTask2();
        s2.studentId = "BAI-002";
        s2.name = "Ali Ishtiaq";
        s2.completedCredits = 60;

        System.out.println("--- Initial State ---");
        System.out.println(s1.summary());
        System.out.println(s2.summary());

        // Add new credits
        s1.addCredits(3);
        s2.addCredits(4);

        System.out.println("\n--- After Adding Credits ---");
        System.out.println(s1.summary());
        System.out.println(s1.name + " Remaining Credits: " + s1.remainingCredits());

        System.out.println("\n" + s2.summary());
        System.out.println(s2.name + " Remaining Credits: " + s2.remainingCredits());
    }
}
```

Prediction:

```
Task 2
=====Before Change=====
credits 30,60
=====After Change=====
credits 33,64
```

Output:

```
C:\Windows\System32\cmd.e  X + v
Microsoft Windows [Version 10.0.26200.9278]
(c) Microsoft Corporation. All rights reserved.

C:\CSC241>javac StudentTask2.java Task2Demo.java

C:\CSC241>java Task2Demoask2
Error: Could not find or load main class Task2Demoask2
Caused by: java.lang.ClassNotFoundException: Task2Demoask2

C:\CSC241>java Task2Demo
--- Initial State ---
ID: BAI-001, Name: Abeer Amina, Completed: 30/130 Credits
ID: BAI-002, Name: Ali Ishtiaq, Completed: 60/130 Credits

--- After Adding Credits ---
ID: BAI-001, Name: Abeer Amina, Completed: 33/130 Credits
Abeer Amina Remaining Credits: 97

ID: BAI-002, Name: Ali Ishtiaq, Completed: 64/130 Credits
Ali Ishtiaq Remaining Credits: 66

C:\CSC241>
```

Concepts Answers:

- **Scope and Lifetime Differences:** An instance variable is declared inside a class (outside methods), exists in heap memory as long as the object exists, and is accessible across all instance methods. A local variable is declared inside a method, exists on the stack only while that method is executing, and is destroyed as soon as the method finishes.
- **Parameter Requirements vs. Instance State:** `remainingCredits(int totalRequired)` needs `totalRequired` as a parameter because total graduation credits vary or are supplied externally at call time, whereas `completedCredits` is already stored directly as part of the object's instance state.

Task 3

Code Picture:

1. Umar tazkeer S | seqdump.txt | seqdump (1).txt | seqdump (2).txt | seqdump (3).txt | seqdump (4).txt | seqdump

File | Edit | View

```
class StudentTask3 {
    String name;

    // Signature: enroll(String)
    void enroll(String courseCode) {
        System.out.println(name + " enrolled in " + courseCode);
    }

    // Signature: enroll(String, int)
    void enroll(String courseCode, int section) {
        System.out.println(name + " enrolled in " + courseCode + " Section " + section);
    }

    // Signature: enroll(int)
    void enroll(int numericCourseCode) {
        System.out.println(name + " enrolled in course number " + numericCourseCode);
    }
}
```

```
class Task3Demo {
    public static void main(String[] args) {
        StudentTask3 st = new StudentTask3();
        st.name = "Abeer Amina";

        // Testing overloaded methods
        st.enroll("CSC241");
        st.enroll("CSC241", 2);
        st.enroll(241);
    }
}
```

Error Code:

```
File Edit View

class StudentTask3 {
    String name;

    // Signature: enroll(String)
    String enroll(String courseCode) {
        System.out.println(name + " enrolled in " + courseCode);
    }

    // Signature: enroll(String, int)
    void enroll(String courseCode, int section) {
        System.out.println(name + " enrolled in " + courseCode + " Section " + section);
    }

    // Signature: enroll(int)
    void enroll(int numericCourseCode) {
        System.out.println(name + " enrolled in course number " + numericCourseCode);
    }
}
```

Prediction:

```
Task 3
=====Before Change=====
All Three Functions will be executed as Java allows you to reuse a method name only if the
list of input parameters (their count or data types) is different.
Abeer Amina enrolled in CSC241
Abeer Amina enrolled in CSC241 Section 2
Abeer Amina enrolled in course number 241

=====After Changes=====
After changing the input parameters of void enroll to " void enroll(String courseCode, int
section)"
It will show error telling that we are repeating the function name and input parameters.
Java did not gave an error to changing return type Because its signature is still
enroll(String). Java ignores the return type (void vs String) when picking a method. If you
were to call st.enroll("CSC241"), Java wouldn't know which of the two enroll(String) methods
to execute!
```

Output:

```

C:\CSC241>javaTask3Demo
'javaTask3Demo' is not recognized as an internal or external command,
operable program or batch file.

C:\CSC241>java Task3Demo
Abeer Amina enrolled in CSC241
Abeer Amina enrolled in CSC241 Section 2
Abeer Amina enrolled in course number 241

C:\CSC241>java Task3Deemo
Error: Could not find or load main class Task3Deemo
Caused by: java.lang.ClassNotFoundException: Task3Deemo

C:\CSC241>java Task3Demo
Abeer Amina enrolled in CSC241
Abeer Amina enrolled in CSC241 Section 2
Abeer Amina enrolled in course number 241

C:\CSC241>javac Task3Demo.java
.\StudentTask3.java:10: error: method enroll(String,int) is already defined
in class StudentTask3
    void enroll(String courseCode, int section) {
        ^
Task3Demo.java:7: error: incompatible types: String cannot be converted to i
nt
        st.enroll("CSC241");
            ^
Note: Some messages have been simplified; recompile with -Xdiags:verbose to
get full output
2 errors

C:\CSC241>java Task3Demo
Abeer Amina enrolled in CSC241
Abeer Amina enrolled in CSC241 Section 2
Abeer Amina enrolled in course number 241

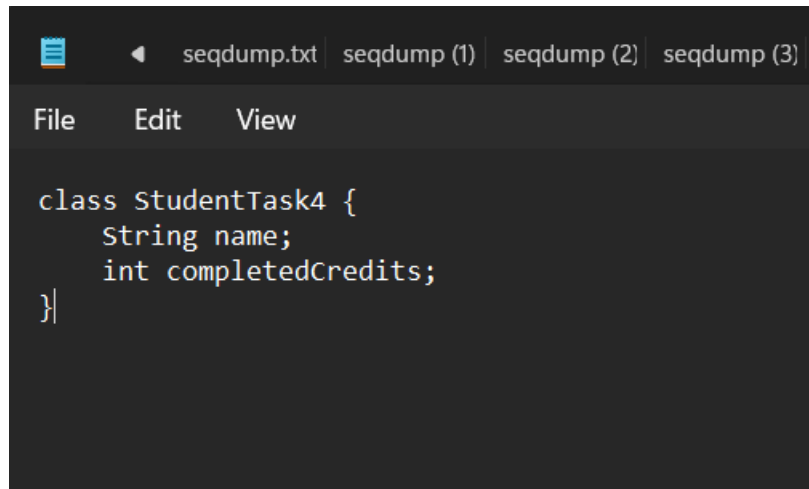
```

Concepts Answers:

- **Method Signature:** enroll(String, int)
- **Overloading Restrictions by Return Type:** The compiler uses only the method name and parameter list (types, count, order) to determine which method to execute. It ignores return types during overload resolution because a method call might not even capture the return value (e.g., st.enroll("CSC241");), leaving the compiler with no way to know which method to call.
- **Compiler Overload Resolution:** The compiler selects enroll(String, int) because the sequence of supplied arguments ("CSC241", 2) matches the (String, int) parameter sequence of that specific method signature.

Task 4

Code Picture:



The image shows a screenshot of a code editor window. The title bar at the top contains a file icon and four tabs: 'seqdump.txt', 'seqdump (1)', 'seqdump (2)', and 'seqdump (3)'. Below the title bar is a menu bar with three items: 'File', 'Edit', and 'View'. The main editing area has a dark background and displays the following C++ code:

```
class StudentTask4 {  
    String name;  
    int completedCredits;  
}
```

File Edit View

```
class PassByValueDemo {

    // Experiment A: Primitive parameter
    void changeNumber(int x) {
        System.out.println("    Inside changeNumber before: " + x);
        x = 99; // Changes local parameter copy
        System.out.println("    Inside changeNumber after: " + x);
    }

    // Experiment B: Object reference parameter (Field Mutation)
    void changeStudent(StudentTask4 st) {
        System.out.println("    Inside changeStudent before: " + st.completedCredits);
        st.completedCredits = 99; // Mutates shared object state
        System.out.println("    Inside changeStudent after: " + st.completedCredits);
    }

    // Experiment C: Object reference parameter (Reassignment)
    void replaceStudent(StudentTask4 st) {
        System.out.println("    Inside replaceStudent before: " + st.name);
        st = new StudentTask4(); // Local reference parameter is reassigned
        st.name = "Temporary";
        System.out.println("    Inside replaceStudent after: " + st.name);
    }

    public static void main(String[] args) {
        PassByValueDemo demo = new PassByValueDemo();

        // --- Experiment A: Primitive ---
        int num = 10;
        System.out.println("--- Experiment A (Primitive) ---");
        System.out.println("Caller Before: " + num);
        demo.changeNumber(num);
        System.out.println("Caller After: " + num);

        // --- Experiment B: Object Mutation ---
        StudentTask4 s1 = new StudentTask4();
        s1.name = "Ali";
        s1.completedCredits = 20;

        System.out.println("\n--- Experiment B (Object Mutation) ---");

        System.out.println("\n--- Experiment B (Object Mutation) ---");
        System.out.println("Caller Before: " + s1.completedCredits);
        demo.changeStudent(s1);
        System.out.println("Caller After: " + s1.completedCredits);

        // --- Experiment C: Object Reassignment ---
        System.out.println("\n--- Experiment C (Object Reassignment) ---");
        System.out.println("Caller Before: " + s1.name);
        demo.replaceStudent(s1);
        System.out.println("Caller After: " + s1.name);
    }
}
```

Prediction:

```
File Edit View H1 Edit B I E A
Task 4
--- TASK 4 PREDICTIONS ---
Experiment A (Primitive):
- Before: 10 | Inside: 99 | After: 10
- Explanation: The argument value is copied into the parameter x, so changing x inside the method does not affect the caller's variable.

Experiment B (Object Mutation):
- Before: 20 | Inside: 99 | After: 99
- Explanation: The argument value is copied into the parameter st, so both variables point to the same object reference, allowing field mutation to affect the caller.

Experiment C (Object Reassignment):
- Before: Ali | Inside: Temporary | After: Ali
- Explanation: The argument value is copied into the parameter st, so assigning st = new StudentTask4() only reassigns the local parameter reference and cannot redirect the caller's reference variable.
```

Output:

```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [Version 10.0.26200.9278]
(c) Microsoft Corporation. All rights reserved.

C:\CSC241>notepad PassByValueDemo.java

C:\CSC241>notepad StudentTask4.java

C:\CSC241>javac StudentTask4.java PassByValueDemo.java

C:\CSC241>java PassByValueDemo
--- Experiment A (Primitive) ---
Caller Before: 10
    Inside changeNumber before: 10
    Inside changeNumber after: 99
Caller After: 10

--- Experiment B (Object Mutation) ---
Caller Before: 20
    Inside changeStudent before: 20
    Inside changeStudent after: 99
Caller After: 99

--- Experiment C (Object Reassignment) ---
Caller Before: Ali
    Inside replaceStudent before: Ali
    Inside replaceStudent after: Temporary
Caller After: Ali

C:\CSC241>
```

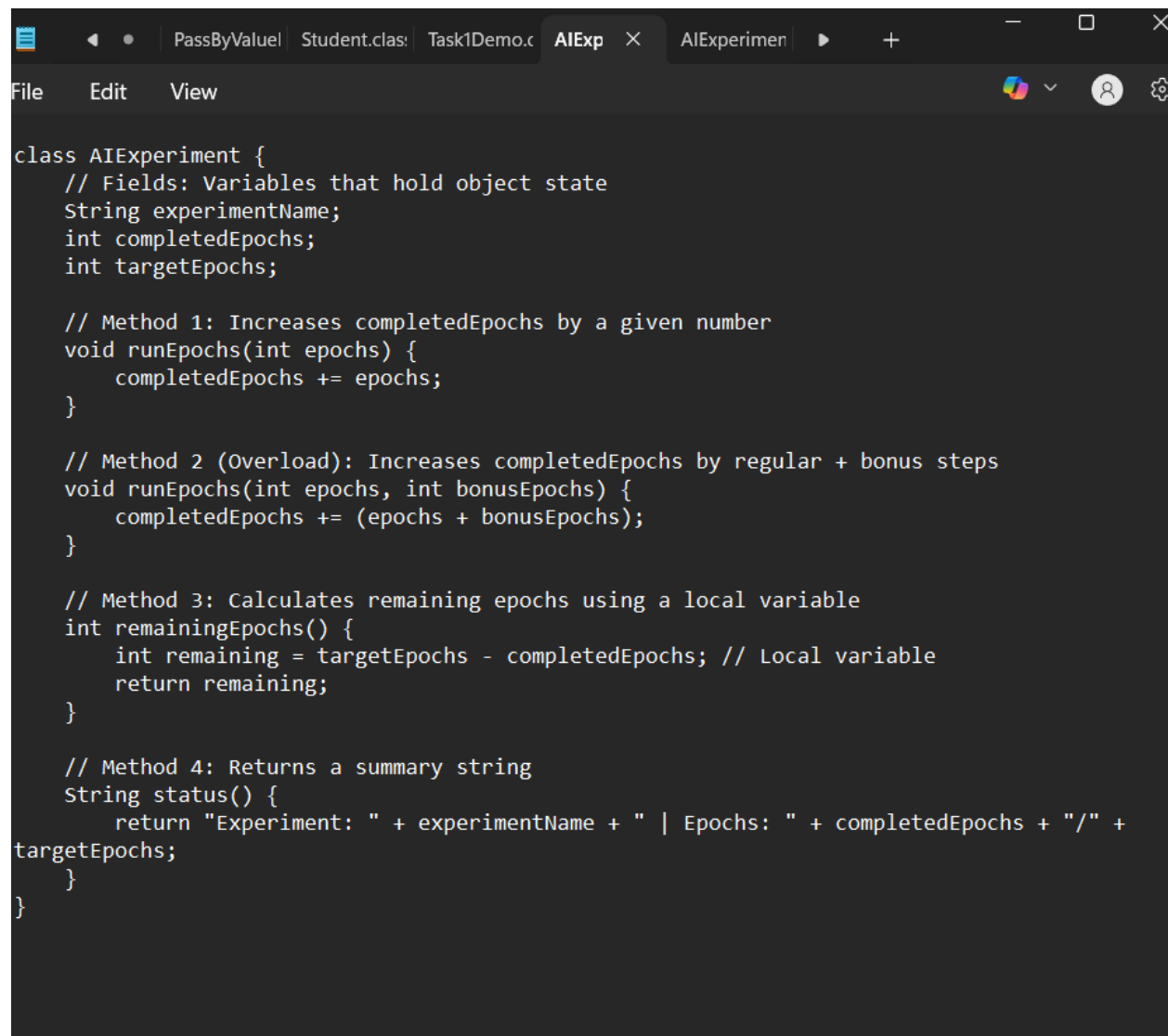
Concepts Answering:

- **Primitive Pass-by-Value Behavior:** The argument value is copied into the parameter `x`, so modifying `x` inside the method only alters that local copy without affecting the caller's original variable.

- **Object Mutation via Copied Reference:** The argument value is copied into the parameter `st`, which means both the caller and the method parameter hold copies of the reference pointing to the exact same object in memory.
- **Object Reassignment via Copied Reference:** The argument value is copied into the parameter, so reassigning `st` to a new object only changes the local parameter reference variable and cannot redirect the caller's original reference variable.

Task 5

Code Picture:



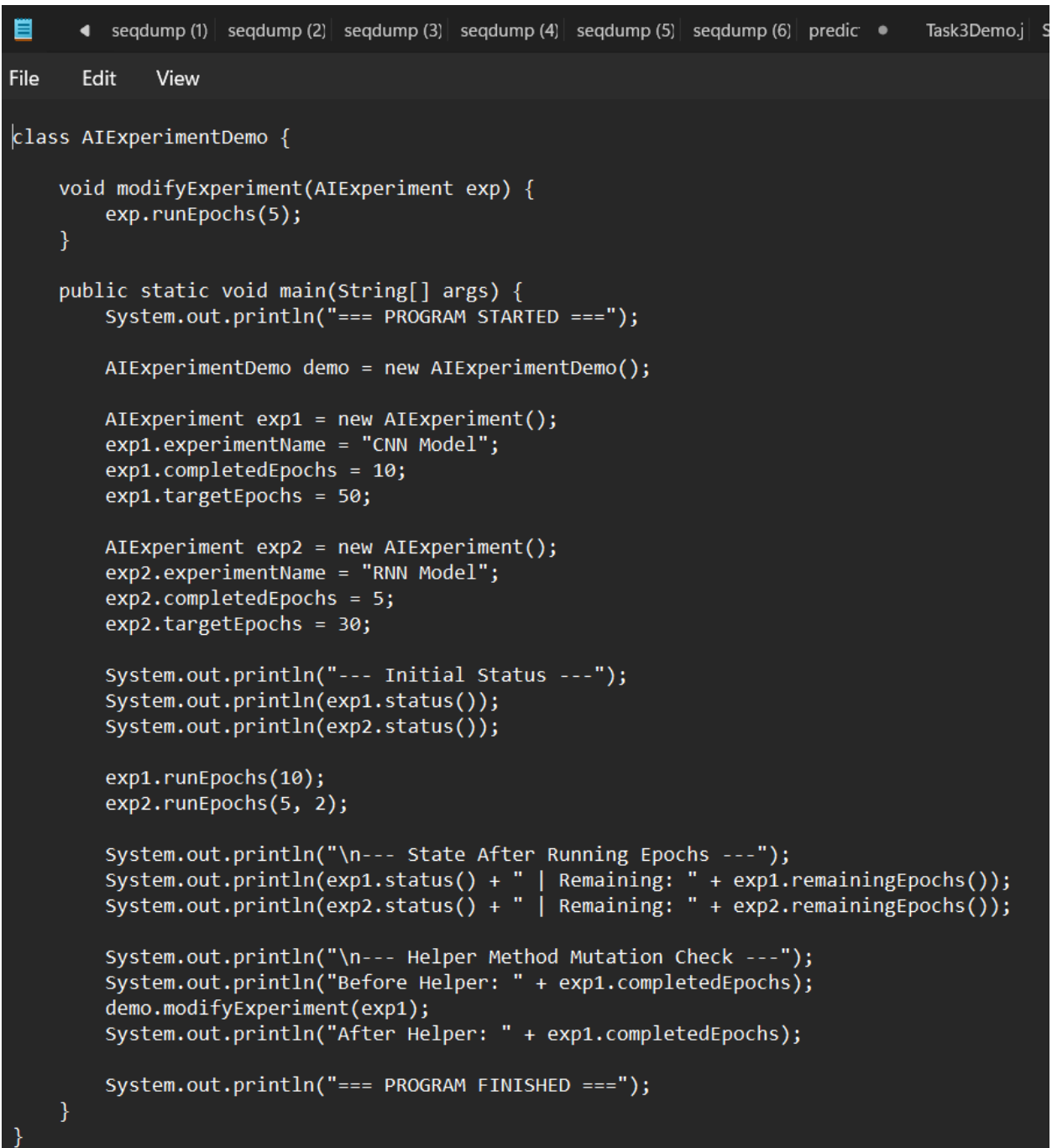
```
class AIExperiment {
    // Fields: Variables that hold object state
    String experimentName;
    int completedEpochs;
    int targetEpochs;

    // Method 1: Increases completedEpochs by a given number
    void runEpochs(int epochs) {
        completedEpochs += epochs;
    }

    // Method 2 (Overload): Increases completedEpochs by regular + bonus steps
    void runEpochs(int epochs, int bonusEpochs) {
        completedEpochs += (epochs + bonusEpochs);
    }

    // Method 3: Calculates remaining epochs using a local variable
    int remainingEpochs() {
        int remaining = targetEpochs - completedEpochs; // Local variable
        return remaining;
    }

    // Method 4: Returns a summary string
    String status() {
        return "Experiment: " + experimentName + " | Epochs: " + completedEpochs + "/" +
targetEpochs;
    }
}
```

The image shows a screenshot of a code editor window. At the top, there is a tab bar with several tabs: 'seqdump (1)', 'seqdump (2)', 'seqdump (3)', 'seqdump (4)', 'seqdump (5)', 'seqdump (6)', 'predic', and 'Task3Demo.j'. The 'Task3Demo.j' tab is currently selected. Below the tab bar, there is a menu bar with 'File', 'Edit', and 'View' options. The main area of the window contains Java code for a class named 'AIExperimentDemo'. The code defines a 'modifyExperiment' method and a 'main' method. The 'main' method initializes two 'AIExperiment' objects, 'exp1' (CNN Model) and 'exp2' (RNN Model), with specific attributes like 'completedEpochs' and 'targetEpochs'. It then prints the initial status, runs a certain number of epochs for each experiment, prints the state after running, and finally prints the status after a helper method 'modifyExperiment' is called on 'exp1'. The program ends with a 'PROGRAM FINISHED' message.

```
class AIExperimentDemo {  
  
    void modifyExperiment(AIExperiment exp) {  
        exp.runEpochs(5);  
    }  
  
    public static void main(String[] args) {  
        System.out.println("=== PROGRAM STARTED ===");  
  
        AIExperimentDemo demo = new AIExperimentDemo();  
  
        AIExperiment exp1 = new AIExperiment();  
        exp1.experimentName = "CNN Model";  
        exp1.completedEpochs = 10;  
        exp1.targetEpochs = 50;  
  
        AIExperiment exp2 = new AIExperiment();  
        exp2.experimentName = "RNN Model";  
        exp2.completedEpochs = 5;  
        exp2.targetEpochs = 30;  
  
        System.out.println("--- Initial Status ---");  
        System.out.println(exp1.status());  
        System.out.println(exp2.status());  
  
        exp1.runEpochs(10);  
        exp2.runEpochs(5, 2);  
  
        System.out.println("\n--- State After Running Epochs ---");  
        System.out.println(exp1.status() + " | Remaining: " + exp1.remainingEpochs());  
        System.out.println(exp2.status() + " | Remaining: " + exp2.remainingEpochs());  
  
        System.out.println("\n--- Helper Method Mutation Check ---");  
        System.out.println("Before Helper: " + exp1.completedEpochs);  
        demo.modifyExperiment(exp1);  
        System.out.println("After Helper: " + exp1.completedEpochs);  
  
        System.out.println("=== PROGRAM FINISHED ===");  
    }  
}
```

Prediction:

```
Task 5
1. Initial State:
- exp1 (CNN Model) will display Epochs: 10/50.
- exp2 (RNN Model) will display Epochs: 5/30.

2. Independent State & Overload Execution:
- exp1.runEpochs(10) will call runEpochs(int) and increase exp1.completedEpochs from 10 to 20.
- exp2.runEpochs(5, 2) will call the overloaded runEpochs(int, int) and increase exp2.completedEpochs by (5 + 2) = 7, raising it from 5 to 12.
- exp1 and exp2 operate independently, so modifying exp1 will not affect exp2.

3. Remaining Epochs Calculation:
- exp1.remainingEpochs() will return 30 (50 - 20).
- exp2.remainingEpochs() will return 18 (30 - 12).

4. Helper Method Pass-By-Value Mutation:
- Calling modifyExperiment(exp1) will pass a copy of the object reference into the parameter exp.
- Calling exp.runEpochs(5) inside the helper method will mutate the shared object, increasing exp1.completedEpochs from 20 to 25.
```

Output Picture:

```
C:\Windows\System32\cmd.e  X  +  v
C:\CSC241>notepad AIExperiment.java
C:\CSC241>notepad AIExperimentDemo.java
C:\CSC241>javac AIExperiment.java AIExperimentDemo.java
C:\CSC241>java AIExperiment.java
error: can't find main(String[]) or main() method in class: AIExperiment
C:\CSC241>java AIExperimentDemo
C:\CSC241>
C:\CSC241>java AIExperimentDemo
C:\CSC241>javac AIExperimentDemo.java
C:\CSC241>java AIExperimentDemo
=== PROGRAM STARTED ===
--- Initial Status ---
Experiment: CNN Model | Epochs: 10/50
Experiment: RNN Model | Epochs: 5/30

--- State After Running Epochs ---
Experiment: CNN Model | Epochs: 20/50 | Remaining: 30
Experiment: RNN Model | Epochs: 12/30 | Remaining: 18

--- Helper Method Mutation Check ---
Before Helper: 20
After Helper: 25
=== PROGRAM FINISHED ===
```

Concepts Answers:

- **Instance Variables vs. Local Variables:** Persistent state properties like `experimentName`, `completedEpochs`, and `targetEpochs` must be stored as instance variables. Temporary calculations like `remaining` in `remainingEpochs()` should remain local variables within their respective methods.
- **State Mutation vs. Computation Methods:** `runEpochs(int epochs)` alters object state by mutating `completedEpochs`, whereas `remainingEpochs()` only computes and returns information without modifying any instance fields.

- **Overloaded Signatures and Resolution:** The method signatures are `runEpochs(int)` and `runEpochs(int, int)`. The compiler distinguishes them because they have different parameter counts.